

1 - Section Intro

In this section, we're going to talk about how to set up our development environment. We'll start off setting up our text editor. I'll be using VS Code, however you're free to use whatever you'd like. I do want to go over a couple VS Code extensions though that will help you out when it comes to things like autocomplete and code highlighting.

Then we'll talk about some of the different ways to setup both PHP and Laravel. Obviously you need PHP installed locally to run Laravel. We'll also be using a PostgreSQL database. You can install everything separately but there are also software suites that include all that stuff for you. I'm going to give you a few options and talk about each one, but ultimately I'm going to use a tool called Laravel Herd, which sets up and configures PHP and Laravel in a couple clicks. It's cross platform and I'll have a video showing you how to get setup on both MacOS and Windows.

For those of you that don't want to use Laravel Herd, I'll also be showing you how to install it using the Composer dependency manager. You'll need to install Composer either way, so make sure you watch that video if you're not familiar with it.

2 - Text Editor Setup

As far as text editors and IDEs go, you have a lot of options. I recommend using [Visual Studio Code](#). It is free, open source, and has a large community of developers that contribute to it and just has a ton of features and great extensions. It is also available on Mac, Windows and Linux, so you can use it on any operating system. This is what I will be using throughout the course.

Some other great options are [PHP Storm]](<https://www.jetbrains.com/phpstorm/>) and [Sublime Text](#). These are not free, but Sublime Text does have a free trial.

So get one of those installed on your system.

code Command

One of the first things that I would recommend doing is adding the `code` command to your PATH. This will allow you to open VS Code from the terminal by typing `code .` in the directory that you want to open. This is really handy and will save you a lot of time.

Open VS Code and press `CMD + SHIFT + P` on Mac or `CTRL + SHIFT + P` on Windows to open the command palette. Type "Shell Command: Install 'code' command in PATH" and press enter. This will add the `code` command to your PATH. Now you can open VS Code from the terminal by typing `code ..`

VS Code Extensions

If you are using VS Code as I am, there are some handy extensions that I would suggest right off the bat. You can find these by clicking on the extensions icon on the left side of the editor.

- [PHP Intelephense](#) - This is a PHP code intelligence extension. It will help you with auto-completion, parameter hints, and more.

- [PHP Docblocker](#) - This will allow you to quickly add docblocks, which are multiple lined comments that describe what a function or a class does. I want to stick to writing clean code in this course and this will help us do that by documenting our code.
- [Laravel Snippets](#) - Provides snippets for Laravel 5 and above.
- [Laravel Blade Snippets](#) - Blade is a template engine used with Laravel and as far as I know, Laravel Snippets does not include snippets or highlighting for Blade files. So you definitely want this installed.

There are other extensions that you may like, but these 4 I definitely recommend.

Formatting

As far as formatting goes, you can select "Format on Save" in your settings and it will automatically format your code when you save, which is really helpful. In order for it to work with PHP, I believe that you have to add the following to your main settings file so that it works with Intelephense.

Open your VSCode user settings JSON file by opening the command palette (CMD + SHIFT + P on Mac, CTRL + SHIFT + P on Windows) and typing "Open Settings (JSON)".

```
"[php]": {  
  "editor.formatOnSave": true,  
  "editor.defaultFormatter": "bmewburn.vscode-intelephense-client"  
},
```

Prettier

[Prettier](#) is a great code formatter. Unfortunately it does not work with PHP out of the box, however, there is a plugin that you can install with NPM if you want.

In order to install the plugin, you need to have [Node.js](#) installed. You can download and install it from the website. Once you have it installed, you can open a terminal and run the following command from whichever directory you are working in:

```
npm init -y  
npm i prettier @prettier/plugin-php
```

This is completely optional. I won't be using the Prettier plugin.

3 - Dev Environment Options

There are many ways to get PHP and Laravel setup on your local machine. In this video, we will look at some of the different ways to do so. You are free to use whichever method you prefer. In this course, I'm going to use Laravel Herd, so I'll show you how to get setup with that, but I'm also going to show you how to do a standard install of Laravel using Composer, which is a PHP package manager.

Unless you're already using something else that you're happy with, Laravel Herd is what I would suggest. I think it is the easiest and most convenient way to get both PHP and Laravel setup for MacOS and Windows. Even if you don't want to use the Herd features, it's still the easiest way to get PHP installed and configured. You can avoid the Herd sites folder and control panel altogether if you want and just use the built in Artisan server, which I'll also demonstrate.

Herd comes configured with an NGINX server and a UI to manage your projects. If you want integrated databases like MySQL or Postgres, you do need the pro version, which I think is \$99. However, you don't need it because you just install your database as a standalone, which is what we'll be doing.

In the next couple videos, I will show you how to get setup on both Mac and Windows. Before we do that, I just want to talk about some of the other options that you have.

Manual Installation

Of course, you can manually install PHP by itself. This is the most basic way to install PHP. The upside to this is that you have full control over the installation and can customize it to your liking. The downside is that you don't have the bells and whistles of a software suite. Also, you may run into some issues when it comes to PHP extensions and configuration.

The process of installing PHP is a little different depending on your operating system. You can use Homebrew on Mac, the installer on Windows and a package manager on Linux distros.

Laragon (Windows Only)

If you are on Windows, there is a tool called Laragon that will install PHP, Apache or NGINX, and MySQL for you. This is a great tool for beginners because it's easy to use and you don't have to worry about configuring anything. It's also great for advanced users because it's very powerful and has a lot of features. It's very fast and lightweight and it's easy to manage multiple versions of PHP if you need to. This is what I used before I learned about Herd.

XAMPP (Windows, Mac, Linux)

XAMPP is an older suite that includes PHP, Apache, and MySQL for you. It's similar to Laragon but it's not as powerful. It's also a bit more difficult to use. I would recommend Laragon over XAMPP if you are on Windows. If you are on Mac or Linux, you can use XAMPP as well. Unfortunately, every time I have tried using XAMPP on Mac, I have run into issues but that could just be me.

Laravel Homestead (Mac, Linux)

If you are on Mac or Linux and want to use Laravel, you can use Laravel Homestead. This is a Vagrant box, which is a virtual environment and it includes its own install of PHP, Nginx, and MySQL. It's also obviously optimized for Laravel. The downside is that it's a little more difficult to use and it can take more resources to run. You also need to install Vagrant and VirtualBox. Vagrant is a tool that allows you to create and manage virtual machines. VirtualBox is the software that allows you to run virtual machines. I haven't used this tool very much, so I can't say much about it.

Laravel Valet (Mac Only)

Laravel Valet is a development environment for macOS designed to be a minimal and fast alternative to using a full virtual machine setup. It's provided by Laravel for developers who prefer a more lightweight environment compared to Homestead. Again this tool is only available for Mac.

Docker

Docker is a tool that allows you to run applications in containers. This is a great way to run PHP applications because you can run them in a container and not have to worry about installing PHP on your machine. This is great for development because you can run multiple versions of PHP at the same time. It's also great for production because you can run your application in a container and not have to worry about the environment. However, I definitely would not recommend this for beginners. It's a little more advanced and can be confusing. It's just something to think about for the future.

Required PHP Extensions For Laravel

If you are going to be using Laravel, there are some PHP extensions that you will need enabled. You really only have to worry about this with a manual standalone installation. If you're using Herd or Laragon or something, these are already configured. With a manual install, you'll need to edit your `php.ini` file.

Here are the extensions you will need:

- **bcmath**: Provides support for arbitrary precision mathematics.
- **ctype**: Functions for character class checks and validation.
- **curl**: Enables transferring data with various protocols (e.g., HTTP, FTP).
- **fileinfo**: Detects file types and provides file information.
- **hash**: Provides hashing algorithms for data security (e.g., MD5, SHA-1).
- **mbstring**: Handles multi-byte string operations for different character encodings.
- **openssl**: Provides encryption and secure communication functions.
- **pdo**: Provides a consistent interface for database access using PDO (PHP Data Objects).
- **session**: Manages user sessions and session data.
- **tokenizer**: Provides functions for tokenizing PHP source code.
- **json**: Provides functions for encoding and decoding JSON data.

4 - PHP & Laravel Herd Setup

Now we're ready to get our dev environment setup. As I mentioned in the last video, there are a lot of different ways to install PHP and get Laravel up and running. If you already have a dev environment and PHP 8.3 is installed, you're fine. If not, I would suggest using Laravel Herd. Even if you don't want to use the Herd features, it's still the easiest way to get PHP installed and configured.

Go to <https://herd.laravel.com> and download the installer. Go through the steps. The install and UI is a bit different depending on if you're on MacOS or Windows, but you get the same features.

Once you get through the install, open the control panel and you should see the NGINX server running. You should see options for your PHP version and a button that says "Open Sites". This is where all of your Laravel sites go.

Let's create our Laravel website. If you aren't using Herd and you're still watching this video, I will show you how to install Laravel using Composer in a later video. It's up to you on how you want to setup your environment. This is definitely the easiest.

Click on "Add Site".

Select "New Laravel Project".

Select "No Starter Kit".

For the project name, add "Workopia".

Leave the default selection for path and tests and initialize a Git repo. It's up to you on how you want to handle version control, but if you want to deploy this project you will need to use Git.

Your project has been created and you have easy access to the terminal, Tinker, which is a command line tool. The project path and the URL.

You can also use the `php` command from anywhere on your machine. Open a terminal and type the following:

```
php --version
```

You should see the version number. For me it is 8.3.11.

You can also run the integrated PHP server if you want. Create a new folder called "test" and a file called "index.php" and add the following:

```
<?php echo 'Hello World'; ?>
```

Now in your terminal, in that folder, run the following:

```
php -S localhost:3000
```

Now when you go to `http://localhost:3000` you should see the hello world.

You can stop the server with `Ctrl+C`.

So you see, you could just install Laravel using Composer in any directory and just use the integrated server.

5 - Install Laravel With Composer

I showed you guys how to install Laravel Herd, which makes it easy to quickly setup a Laravel website in a couple clicks. However I also want to show you the standard way to install Laravel with Composer. So if you're not using Herd, you can get setup with this. Even if you are using Herd, you still want to watch this and get Composer installed on your machine.

What Is Composer?

Composer is a package or a dependency manager for PHP. Just about every language has something similar for installing packages. With Node.js, you have NPM (Node Package Manager), with Ruby, you have RubyGems and with Python, you use pip (Python Package Installer). Composer can be used to install all kinds of software, Laravel included. It also offers autoloading of classes and other stuff. So regardless of how you install Laravel, you want to get Composer installed.

Installing Composer

You can visit the official website for Composer at <https://getcomposer.org>. Depending on your OS, there are different ways to install Composer.

Mac Install

For mac, you can use Homebrew. If you don't have Homebrew installed, you can install it by running the following command in your terminal:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

This will install Homebrew on your system.

Install Composer With Homebrew

To install composer on MacOS with Homebrew, simply run the following:

```
brew install composer
```

You can clean up the installation by running:

```
brew cleanup composer
```

To test it out, run the following:

```
composer
```

You should see a list of commands.

Composer is now installed.

Windows Install

Go to the website <https://getcomposer.org> and click on the "Download" button. From here, you will see "Windows Installer". Click the link "Composer-Setup.exe". Once you download the file, run it and go through the installer and just select the defaults.

Now open up a terminal and run `composer`. If it says something like "Command not found", you need to restart your machine. I know this is a bit of a pain.

Now try running `composer` again. You should see a list of commands.

Composer is now installed.

Laravel Installation

To install Laravel, open your terminal in the folder that you want to setup your project in and run the following:

```
composer create-project laravel/laravel workopia
```

This will create a new Laravel project in a folder called `workopia`.

Now navigate to the project folder:

```
cd workopia
```

Open VS Code:

```
code .
```

I

Artisan

Laravel comes with an amazing command-line tool called Artisan. You can use Artisan to run commands to help you with your Laravel project. You can use it to create models, controllers, migrations, etc. You can also use it to run your Laravel project locally.

To see a list of available commands, run:

```
php artisan
```

You will see a bunch of commands including `serve`, `migrate`, `make:model`, `make:controller`, etc. We will be using Artisan a lot in this course.

Running the Project

To run your Laravel project locally, run:

```
php artisan serve
```

This will start a local server at <http://localhost:8000>. You can now visit that URL in your browser to see your Laravel project. You should see the landing page.

 Laravel landing page

We now have Laravel setup and our project running. In the next lesson, we'll take a look at the Laravel folder structure.